

APS Database (aps-db)

PostgreSQL database layer for the Apartment Booking System. This repository contains the database schema, seed dataset, Docker Compose setup, and Windows helper scripts used by the **aps-be** REST versus GraphQL backend experiment.

The database is intentionally separated from the backend so REST and GraphQL can be tested against the same schema and seed data under controlled conditions.

Project Scope

This repository provides:

- PostgreSQL 16 database service.
- pgAdmin web UI for database inspection.
- Ordered SQL migrations for APS users, facilities, bookings, constraints, triggers, and indexes.
- Seed data for local development and benchmark reproduction.
- Windows batch scripts for initializing, resetting, and reseeding the database.

The API implementation and load-test runner live in the sibling **aps-be** project.

Schema Overview

Table	Purpose
users	User accounts, phone-number login identity, roles, tenant verification, and unit numbers
facilities	Bookable apartment facilities, pricing, operating hours, and active status
bookings	Facility reservations, status, date and time range, price, and relationships to users and facilities

Important schema behavior:

- **user_role** enum: **guest**, **tenant**, **admin**, **super_admin**.
- **booking_status** enum: **pending**, **confirmed**, **expired**, **cancelled**.
- Foreign keys prevent deleting users or facilities that are still referenced by bookings.
- CHECK constraints validate phone numbers, tenant unit requirements, non-negative prices, valid operating hours, booking time ranges, and maximum advance booking date.
- **updated_at** timestamps are maintained by triggers.
- Indexes support login lookup, role filters, active facility lookup, booking history, availability checks, conflict detection, and daily booking-count validation.
- A partial unique index prevents duplicate active bookings for the same facility, date, start time, and end time while excluding **cancelled** and **expired** bookings.

Seed Dataset

The seed files create a reproducible local dataset for development and load testing:

- `seeds/001_users.sql` truncates and loads administrative users, tenants, guests, and generated benchmark users.
- `seeds/002_facilities.sql` truncates and loads seven facilities: Tennis Court, Function Hall, Ballroom, Meeting Room A, Meeting Room B, Swimming Pool, and Gym.
- `seeds/003_bookings.sql` truncates and generates historical booking records, then calculates prices based on tenant status and facility pricing.

The backend k6 tests depend on the seeded phone numbers and user IDs defined here. Keep this dataset stable when comparing REST and GraphQL results across runs.

Prerequisites

- Docker Desktop or Docker Engine.
- Docker Compose v2 (`docker compose`) or the legacy `docker-compose` command.
- Optional: `psql`, DBeaver, DataGrip, or another PostgreSQL client.

Environment

The Compose file reads configuration from `.env`. The local development defaults are:

```
POSTGRES_PORT=5432
POSTGRES_USER=apsadmin
POSTGRES_DB=apartment_booking
POSTGRES_PASSWORD=adminpassword
POSTGRES_HOST=postgres

PGADMIN_PORT=5050
PGADMIN_DEFAULT_EMAIL=admin@gmail.com
PGADMIN_DEFAULT_PASSWORD=adminpassword
```

These values are suitable for a local research environment only. Replace passwords before using the setup anywhere outside local development.

Quick Start

From the `aps-db` directory:

```
docker compose up -d
docker compose ps
```

On first initialization, PostgreSQL executes the mounted files in alphabetical order:

```
001_create_users_table.sql
002_create_facilities_table.sql
003_create_bookings_table.sql
004_create_indexes.sql
```

```
005_seed_users.sql
006_seed_facilities.sql
007_seed_bookings.sql
```

Because the seed files are currently mounted in `docker-compose.yml`, a new database volume is created with both schema and seed data already loaded.

Verify the database:

```
docker compose exec postgres psql -U apsaadmin -d apartment_booking -c "\dt"
docker compose exec postgres psql -U apsaadmin -d apartment_booking -c
"SELECT COUNT(*) FROM users;"
docker compose exec postgres psql -U apsaadmin -d apartment_booking -c
"SELECT COUNT(*) FROM facilities;"
docker compose exec postgres psql -U apsaadmin -d apartment_booking -c
"SELECT COUNT(*) FROM bookings;"
```

Accessing PostgreSQL

Use `psql` through Docker:

```
docker compose exec postgres psql -U apsaadmin -d apartment_booking
```

Connection details for external tools:

```
Host: localhost
Port: 5432
Database: apartment_booking
Username: apsaadmin
Password: adminpassword
```

For backend containers on the Docker network, use:

```
Host: aps-postgres
Port: 5432
Database: apartment_booking
Username: apsaadmin
Password: adminpassword
```

pgAdmin

Open pgAdmin at:

```
http://localhost:5050
```

Login:

```
Email: admin@gmail.com
Password: adminpassword
```

Register the PostgreSQL server with:

```
Host name/address: postgres
Port: 5432
Maintenance database: apartment_booking
Username: apsadmin
Password: adminpassword
```

Windows Helper Scripts

The `scripts/` directory contains Windows batch helpers:

Script	Purpose
<code>scripts\init-db.bat</code>	Runs all files in <code>migrations*.sql</code> against the running <code>aps-postgres</code> container
<code>scripts\seed-db.bat</code>	Runs all files in <code>seeds*.sql</code> and prints a summary
<code>scripts\reset-db.bat</code>	Drops and recreates the configured database, then instructs you to rerun migrations and seeds

Run these from the `aps-db` directory after the PostgreSQL container is running.

Important: the seed files use `TRUNCATE ... CASCADE` and are intended for local/test data. Do not run them against a database whose data must be preserved.

Resetting the Database

The most reliable full reset is to remove the Docker volume and start again:

```
docker compose down -v
docker compose up -d
```

This destroys all existing database and pgAdmin volume data, then replays migrations and mounted seeds from scratch.

On Windows, the helper workflow is:

```
scripts\reset-db.bat
scripts\init-db.bat
scripts\seed-db.bat
```

Use the Docker volume reset when you want the exact same first-start behavior as a new machine.

Relationship to aps-be

The backend Compose file in **aps-be** joins the external Docker network created by this database project:

```
aps-db_aps-network
```

Start this database project before starting **aps-be**:

```
cd aps-db
docker compose up -d

cd ../aps-be
docker compose up -d --build
```

The backend **.env** should point to:

```
DB_HOST=aps-postgres
DB_PORT=5432
DB_USER=apsadmin
DB_PASSWORD=adminpassword
DB_NAME=apartment_booking
DATABASE_URL=postgresql://apsadmin:adminpassword@aps-
postgres:5432/apartment_booking
```

Useful Queries

User role distribution:

```
SELECT role, is_verified_tenant, COUNT(*)
FROM users
GROUP BY role, is_verified_tenant
ORDER BY role, is_verified_tenant;
```

Booking status distribution:

```
SELECT status, COUNT(*)
FROM bookings
GROUP BY status
ORDER BY status;
```

Facility list:

```
SELECT id, name, price_per_hour, open_time, close_time, is_active
FROM facilities
ORDER BY id;
```

Detect overlapping active bookings:

```
SELECT
    b1.facility_id,
    b1.booking_date,
    b1.start_time,
    b1.end_time,
    b2.start_time,
    b2.end_time
FROM bookings b1
JOIN bookings b2
    ON b1.facility_id = b2.facility_id
   AND b1.booking_date = b2.booking_date
   AND b1.id <> b2.id
WHERE b1.status NOT IN ('cancelled', 'expired')
   AND b2.status NOT IN ('cancelled', 'expired')
   AND (b1.start_time, b1.end_time) OVERLAPS (b2.start_time, b2.end_time);
```

Use this query as a diagnostic when validating the seed data. The database-level unique index prevents exact duplicate active slots; overlapping ranges with different start and end times should be evaluated according to the experiment assumptions and application-level booking rules.

Directory Structure

```
aps-db/
  migrations/
    001_create_users_table.sql
    002_create_facilities_table.sql
    003_create_bookings_table.sql
    004_create_indexes.sql
  seeds/
    001_users.sql
    002_facilities.sql
    003_bookings.sql
```

```
scripts/  
  init-db.bat  
  reset-db.bat  
  seed-db.bat  
docker-compose.yml  
.env  
README.md
```

Troubleshooting

Check running services:

```
docker compose ps
```

Read PostgreSQL logs:

```
docker compose logs postgres
```

Confirm tables exist:

```
docker compose exec postgres psql -U apsaadmin -d apartment_booking -c "\dt"
```

If port **5432** is already used by another local PostgreSQL installation, change **POSTGRES_PORT** in **.env** and restart the Compose stack.

If migrations or seed files do not appear to run, remember that **/docker-entrypoint-initdb.d** files only run automatically when the PostgreSQL data volume is empty. Use **docker compose down -v** before recreating the database from scratch.